

Professional Certificate Programme in Agentic AI and Applications



Working with Libraries



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- An introduction to Python fundamentals, including variables, data types and numerical operations
- Understanding Python data structures as essential building blocks for efficient programming
- How lists store ordered collections and support flexible data manipulation
- How tuples provide fixed, immutable sequences for secure data grouping
- How sets enable fast membership checks and the removal of duplicate values
- How dictionaries store key–value pairs for structured and accessible data handling
- How Python conditional statements control the flow of logic based on different conditions

Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- How loops in Python automate repetitive tasks and streamline workflows
- How to build a simple chatbot in Python using core programming techniques
- A recap of essential Python concepts to reinforce understanding
- How user-defined functions improve modularity, reuse and clarity in Python programmes
- An applied example of integrating functions into a Python-based chatbot

Topics Covered

- Error Handling in Python
- Using Python Libraries: Standard & Third-Party Modules in Action
- Importing Modules
- Standard Libraries
- Building CLI Calculator

Topics Covered

- Project Structure
- Working with NumPy and Pandas
- Python
- Data Visualisation with Matplotlib and Seaborn
- Mastering Code Modularisation

Error Handling in Python

Error Handling in Python

Key Objectives

```
237     if access_type == WinregRegistryEntry.KEY_WOW64_64:  
238         raise RuntimeError("Use either KEY_WOW64 or KEY_WOW32 with create_value()")  
239  
240     registry_key = None  
241     wow64_flags = WOW64_MAP[access_type]  
242     try:  
243         key_hive_value = HIVES_MAP[key_hive]  
244  
245         if isinstance(value_type, RegistryKeyType):  
246             value_type = TYPES_MAP[value_type]  
247  
248         registry_key = winreg.OpenKey(key_hive_value, key_path, 0, (wow64_flags | winreg.  
249         winreg.SetValueEx(registry_key, value_name, 0, value_type, key_value)  
250         winreg.CloseKey(registry_key)  
251         return True  
252     except Error as e:  
253         logger.error("Unable to write to registry path %s\\%s with LastError=%d [%s]",  
254             key_hive, key_path, e.winerror, e.strerror)  
255         if registry_key is not None:  
256             winreg.CloseKey(registry_key)  
257         return False  
  
def delete_value(key_hive, key_path, value_name, access_type=WinregRegistryEntry.KEY_WOW64_64):  
    """  
    Delete registry value  
    :param key_hive: Windows registry hive to edit, e.g. HKEY_CURRENT_USER  
    :param key_path: Path Windows registry key should live under, for example "SOFTWARE\\  
    """
```

- Understand errors and exceptions to write safer, more robust code
- Apply practical strategies for effective error handling in Python

Importance of Error Handling in Python

Without Handling	With Handling
Program crashes abruptly, frustrating users and risking data loss	Program responds gracefully, provides feedback and continues running

Types of Errors

Syntax errors

```
print("Hello" # Missing closing parenthesis
```

Occur due to code structure mistakes and are caught before execution

Runtime errors (Exceptions)

```
result = 10 / 0 # ZeroDivisionError
```

Happen during execution when unexpected events occur and must be handled

Common Exceptions in Python

ZeroDivisionError

Occurs when dividing by zero: `10 / 0`

IndexError

List index out of range: `my_list[99]`

ValueError

Invalid value: `int("hello")`

KeyError

Dictionary key not found:
`my_dict["missing"]`

TypeError

Operation on incompatible types: `"2" + 2`

FileNotFoundError

Missing file: `("nonexistent.txt")`

The try-except Block

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")

except ValueError:
    print("Error: Please enter a valid number.")
```

try:

Contains the code that might cause an exception

except

Specifies what to do when specific errors occur

The else Clause

```
try:
    num = int(input("Enter a number: "))    # May cause ValueError
except ValueError:
    print("Invalid input! Please enter a valid number.")
else:
    print("You entered:", num)
    # More operations with num
```

Purpose: Runs code only if the try block executes without exceptions

Benefit: Separates normal flow from error handling for clearer, more maintainable code

The finally Clause

```
try:
    f = open("data.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File not found!")
finally:
    try:
        f.close()
    except NameError:
        pass # f was never opened
```

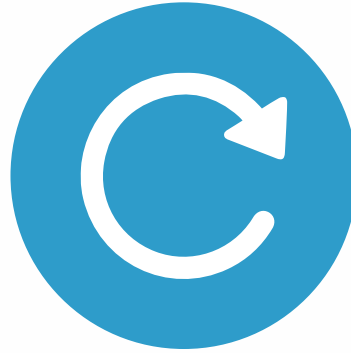
Executes code regardless of whether an exception occurs

The finally Clause



Resource cleanup

Close files, database connections and network sockets



State reset

Resets application state or flags after operations



Logging

Logs whether an operation completed or failed

Raising Exceptions

```
def check_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative")  
    if age > 120:  
        raise ValueError("Age unrealistically high")  
    print("Valid age:", age)  
  
try:  
    check_age(-5)  
except ValueError as error:  
    print(error) # Output: Age cannot be negative
```

Use **raise** to trigger custom exceptions when invalid conditions occur.

Raising Exceptions

Use raise to



Enforce business
rules



Validate input data



Signal impossible
conditions

Best Practices

Be specific

Catch specific exceptions (`ValueError`, `ZeroDivisionError`), not just bare `except`: which can hide bugs

Keep try blocks small

Wrap only the lines that may raise exceptions, not large code blocks

Provide helpful messages

Give error messages that guide users to resolve issues

Clean up resources

Use `finally` or context managers (`with` statement) for cleanup

Using Python Libraries

Standard & Third-Party Modules in Action

Python Libraries

Pre-Written Code for Ready-to-Use Functionality



Saves time

Use existing solutions instead of building from scratch



Reduces errors

Leverage thoroughly tested code from experts

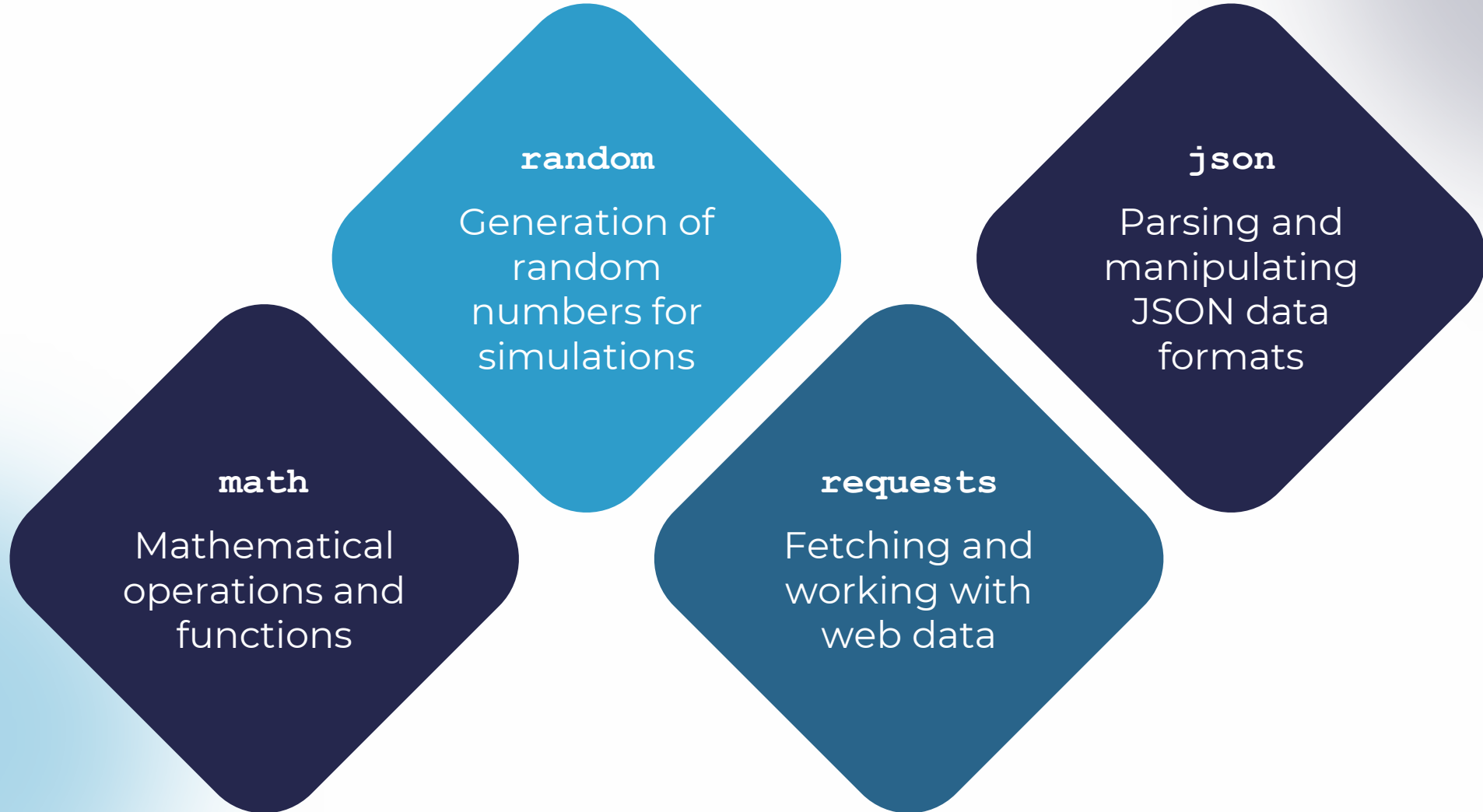


Encourages reuse

Promote modular programming practices

Python Libraries

Common Examples



Importance of Python Libraries

Extend functionality with Python's vast collection of pre-written, tested libraries

Use existing solutions to avoid reinventing the wheel

Focus on solving problems instead of writing infrastructure code from scratch

Access a rich ecosystem



Save time



Improve productivity



Two Types of Python Libraries

Standard libraries	Third-party libraries
Included with Python by default	Require separate installation (commonly via pip)
Available for use without setup	Add functionality beyond Python's core features

Types of Python Libraries

Key Functionalities

Standard libraries (Built-in with Python, no setup required)	Third-party libraries (Community-created and installed via pip)
math: Advanced mathematical functions	requests: HTTP library for API calls
random: Pseudo-random number generators	numpy: Scientific computing
datetime: Date and time handling	pandas: Data analysis and manipulation
os: Operating system interfaces	matplotlib: Data visualisation

Importing Modules

Importing Modules — Basics

- Use import to bring in the whole module

```
import math  
print(math.sqrt(16)) # Output: 4.0
```

- Access functions with `module_name.function()`
- Keep namespace clean and organised

Importing with Aliases

Use Shorter Aliases

```
import math as m  
print(m.sqrt(25)) # Output: 5.0
```



Import modules with `as` for concise code and clear namespace



Follow community naming conventions

Importing with Aliases

Common Data Science Aliases

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
```

These aliases are widely recognised by experienced Python developers.

Importing Specific Functions

from Syntax

```
from math import sqrt, pi
print(sqrt(36)) # Output: 6.0
print(pi)       # Output: 3.141592...
```

- Import only needed items
- Access functions without the module prefix
- Keep code cleaner when using few functions

Importing Specific Functions

When to Use This Approach



Import only a few specific functions



Ensure function names are distinctive to avoid conflicts



Improve readability by omitting the module prefix

Import All (Not Recommended)

```
from math import *  
print(sqrt(49)) # Output: 7.0
```

Use asterisk carefully: Imports all items from a module into the current namespace

Import All (Not Recommended)

Risks of Using * Imports



Triggers unexpected name conflicts



Obscures the origin of functions



Clutters the namespace with unused functions



Complicates debugging and maintenance

Best practice: Always specify what to import for maintainable code.

Standard Libraries

Standard Library — math

Providing standard C-based mathematical functions

```
import math

print(math.sqrt(64))      # Output: 8.0
print(math.pi)           # Output: 3.141592653589793
print(math.factorial(5))  # Output: 120
```

Provides standard C-based mathematical functions

Standard Library — math

Common Functions

`math.sqrt(x)` - Square root

`math.factorial(x)` - Factorial

`math.pi`, `math.e` - Constants

`math.pow(x, y)` - Power function

`math.ceil(x)`, `math.floor(x)` - Rounding

`math.sin(x)`, `math.cos(x)` - Trigonometry

Standard Library — random

```
import random

# Random integer between 1 and 10
print(random.randint(1, 10))

# Random item from a list
print(random.choice(["apple", "banana", "cherry"]))

# Random float between 0 and 1
print(random.random())
```

Implements pseudo-random number generators for multiple distributions

Standard Library — random

Use Cases



Used in games and simulations



Supports statistical sampling



Provides random testing data



Assists in algorithm testing

Third-Party Library — requests

Install first: `pip install requests`

```
import requests

# Get data from GitHub API
response = requests.get("https://api.github.com")

# Check if request was successful
print(response.status_code) # 200 means OK

# Access JSON response data
data = response.json()
print(data)
```

The requests library simplifies making HTTP requests and handling responses.

Third-Party Library — requests

Key Features



Provides an easy-to-use interface for HTTP methods (GET, POST, etc.)



Handles JSON parsing automatically



Maintains sessions and manages cookies



Supports authentication mechanisms



Enables file uploads and downloads

Third-Party Library — json

```
import json

# Parse JSON string to Python dictionary
person = '{"name": "Amit", "age": 30}'
data = json.loads(person)
print(data["name"]) # Output: Amit

# Convert Python dictionary to JSON string
dict_obj = {"city": "Hyderabad", "country": "India"}
json_str = json.dumps(dict_obj)
print(json_str) # {"city": "Hyderabad", "country": "India"}
```

Third-Party Library — json

Key Functions



`json.loads()` - Parses JSON string to Python object



`json.dumps()` - Converts Python object to JSON string



`json.load()` - Reads JSON from a file



`json.dump()` - Writes Python object as JSON to a file

Combining Libraries

Example

```
import requests
import json

# Fetch data from a public API
url = "https://jsonplaceholder.typicode.com/todos/1"
response = requests.get(url)

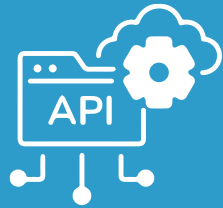
# Parse the JSON response
data = json.loads(response.text)

# Print the title from the response
print(data["title"])
```

Requests retrieves data,
and JSON converts it into
a Python dictionary

Combining Libraries

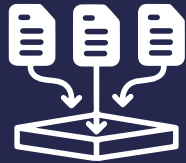
Real World Applications



Develop API
integrations



Create web
applications



Construct data
pipelines



Web scraping
& automation

Best Practices

Import organisation

- Group standard libraries first, third-party libraries next and local modules last
- Separate groups with a blank line for readability

Import efficiency

- Import only what you need
- Use aliases for verbose module names
- Avoid wildcard imports (`from x import *`)

Best Practices

Code organisation: Structure imports consistently

```
# Standard libraries
import os
import math

# Third-party libraries
import requests
import pandas as pd

# Local modules
import my_module
from my_package import my_function
```

Examples

Calculate Circle Area

Uses the `math` module to calculate the area of a circle with mathematical precision.

```
import math

radius = 5
area = math.pi * radius ** 2
print(f"Area: {area:.2f}") # Output: Area: 78.54
```


Examples

Simulate Dice Roll

Uses the random module to create a versatile dice-rolling function.

```
import random

def roll_dice(sides=6):
    return random.randint(1, sides)

print(roll_dice())      # Random number between 1-6
print(roll_dice(20))    # Random number between 1-20
```

Examples

Fetch Weather Data

Combines `requests` and `json` to fetch real weather data.

```
import requests
import json

url = "https://api.openweathermap.org/data/2.5/weather"
params = {
    "q": "New York",
    "appid": "YOUR_API_KEY" # Replace with your actual API key
}

response = requests.get(url, params=params)
data = json.loads(response.text)

# Convert temperature from Kelvin to Celsius
temp = data["main"]["temp"] - 273.15
print(f"{temp:.1f}°C")
```

Building a CLI Calculator

Significance of CLI Calculator

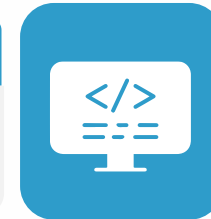
Apply functions

Practice reusable logic through functions



Use modularisation

Organise code by separating it across multiple files



Handle user input/output

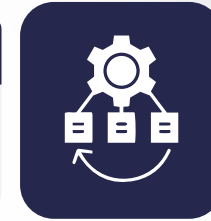
Manage interactions via the command-line interface



Significance of CLI Calculator

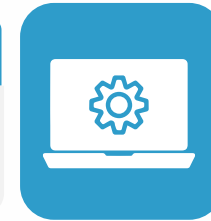
Encourage clean project structure

Develop good coding habits for code organisation



Mimic real-world practices

Experience how professional software is structured



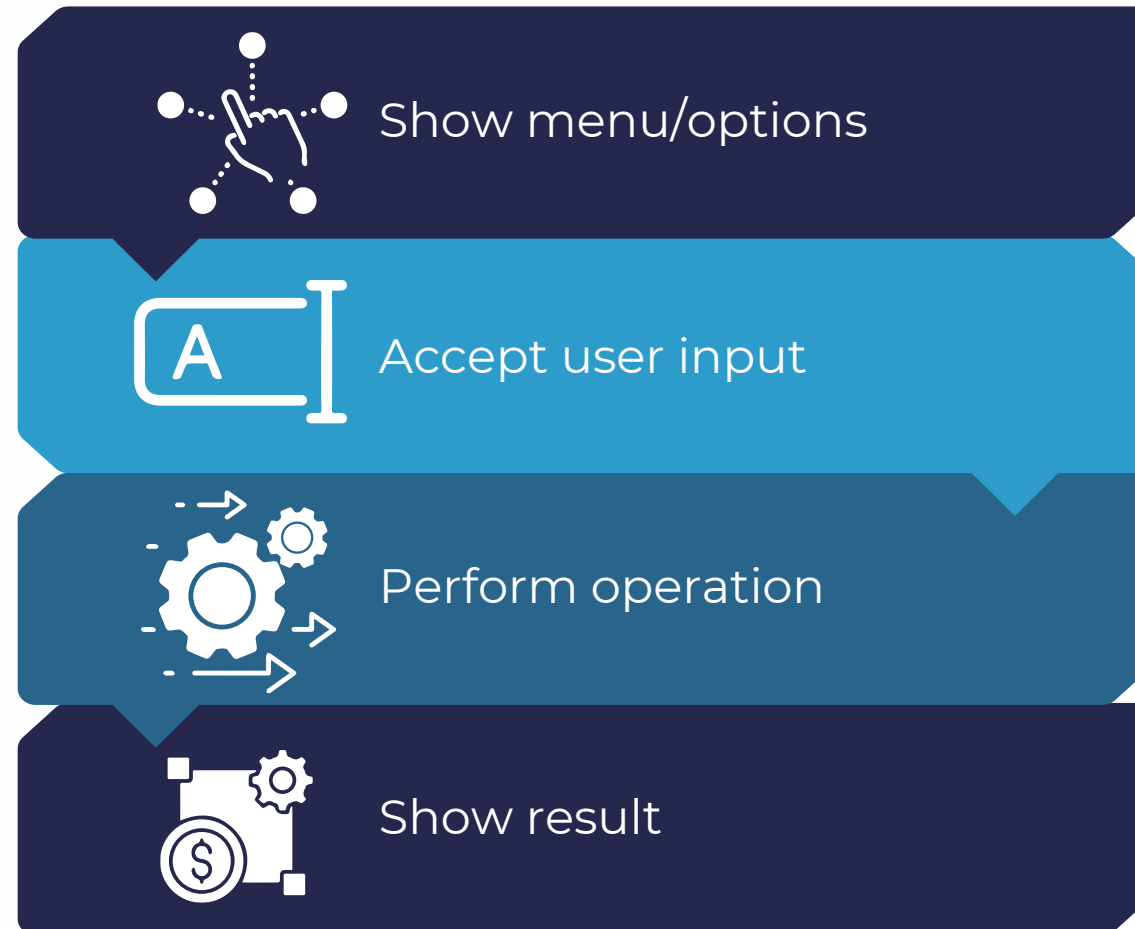
CLI – Command Line Interface



- Runs programs directly in the terminal or console
- Interacts using text only, without a GUI

CLI – Command Line Interface

Key Steps Involved



Project Structure

Project Structure

```
calculator_project/  
|  
├─ main.py          # Entry point (menu, input/output)  
├─ operations.py    # Functions: add, subtract, multiply, divide  
└─ utils.py         # Helper functions (validate input, etc.)
```

operations.py (Functions)

```
# operations.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b
```

Each function performs a specific operation and returns the result.

utils.py (Helper Functions)

```
def get_number(prompt):  
    """Prompt the user for a number until valid input is provided."""  
    while True:  
        try:  
            return float(input(prompt))  
        except ValueError:  
            print("Invalid input! Please enter a number.")
```

- **Prompt the user:** Display a message requesting numeric input.
- **Validate the input:** Convert input to float using try-except.
- **Return or retry:** Return valid number or show error and repeat.

main.py (CLI Entry Point)

```
from operations import add, subtract, multiply, divide
from utils import get_number

while True:
    print("\n--- CLI Calculator ---")
    print("1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Exit")

    choice = input("Enter choice: ")

    if choice == "5":
        print("Goodbye!")
        break
```

```
x = get_number("Enter first number: ")
y = get_number("Enter second number: ")

if choice == "1":
    print("Result:", add(x, y))
elif choice == "2":
    print("Result:", subtract(x, y))
elif choice == "3":
    print("Result:", multiply(x, y))
elif choice == "4":
    print("Result:", divide(x, y))
else:
    print("Invalid choice!")
```

Running the Calculator

Command in terminal:

```
python main.py
```

Sample Run:

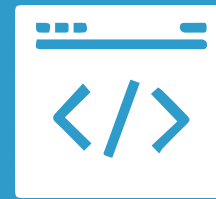
```
--- CLI Calculator ---  
1. Add  
2. Subtract  
3. Multiply  
4. Divide  
5. Exit  
  
Enter choice: 1  
Enter first number: 10  
Enter second number: 5  
Result: 15
```

Key Takeaways



Enable code reuse

Write functions once and use them multiple times



Practice real-world projects

Develop CLI applications to build skills for larger systems



Improve organisation

Split code across files for easier maintenance and clarity



Support scalability

Add new functions without modifying the core project structure

Hands-on Call a public API

Refer to exercise document.

Working with NumPy and Pandas

A practical introduction to efficient data analysis using Python's powerful numerical libraries

Numpy & Pandas

Lists and dictionaries are flexible but inefficient for large data analysis

NumPy	Pandas
Performs fast array operations	Built on NumPy for efficiency
Serves as the foundation for scientific computing	Handle tabular data easily
Benefits from optimised C implementation	Offer Excel-like functionality with programmability

Importing Libraries

```
import numpy as np  
import pandas as pd
```

Use standard aliases: np for NumPy and pd for Pandas are universally recognised

Place Imports correctly: Place imports at the top of the file

NumPy Basics

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))
```

Performance: 10–100x faster than Python lists for mathematical operations

Functionality: 50+ built-in mathematical and statistical functions

NumPy Basics

Example Outputs

```
[1 2 3 4 5]
```

```
Mean: 3.0
```

```
Standard Deviation: 1.414
```

Python

Pandas Dominating Data Science

Key Reasons



Efficient processing

Handle large
structured datasets
quickly



Seamless integration

Work smoothly
with NumPy,
Matplotlib, and
Scikit-Learn



Flexible & expressive

Use intuitive syntax
for clear and
simple data
manipulation

Essential Data Analysis Functions

Data inspection

Use `.head()`, `.tail()`, `.info()` and `.shape` to examine structure and quality

Sorting & filtering

Organise with `.sort_values()` and filter using `.query()` or conditions

Smart selection

Access data with `.loc[]`, `.iloc[]` and boolean indexing for precise analysis

Data quality

Remove duplicates with `.drop_duplicates()` to ensure data integrity

Pandas Data Structures

```
import pandas as pd

s1 = pd.Series([12, 34, 56, 78], name="values")
print(s1)
```

```
import pandas as pd

data = {
    "Name": ["Amit", "Priya"],
    "Age": [30, 25]
}
df = pd.DataFrame(data)
print(df)
```

Series

- Represents a one-dimensional labelled array
- Stores values with an index for easy access

DataFrame

- Represents a two-dimensional labelled table
- Combines multiple Series objects with shared index

Loading CSV with Pandas

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head()) # prints first 5 rows
```

- `pd.read_csv()` → loads CSV files into DataFrame
- `df.head()` → quick preview of data (first 5 rows)
- `df.head(10)` → view first 10 rows

Loading CSV with Pandas

Pandas also supports Excel, JSON, SQL and other formats via similar functions

```
pd.read_excel('file.xlsx') # load Excel file  
pd.read_json('file.json')  # load JSON file  
pd.read_sql('SELECT * FROM table', connection) # load from SQL
```

Selecting Data



Single column

Use `print(df['Name'])` to return a Series object



Multiple columns

Use `print(df[['Name', 'Age']])` (note the double brackets) to return a DataFrame



Row by position

Use `print(df.iloc[0])` for integer location-based indexing



Specific cell

Use `print(df.loc[0, 'Name'])` for label-based indexing

Inspecting Data



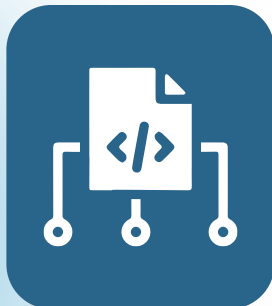
Size

- Use `print(df.shape)` to return (rows, columns)
- Example: `(891, 12)`



Column names

- Use `print(df.columns)` to show all column headers
- Example: `Index(['Name', 'Age', ...])`



Metadata

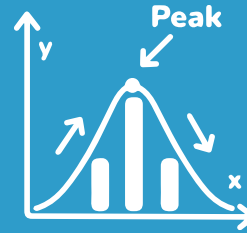
- Use `print(df.info())` to view data types, non-null values and memory usage

Data Summarisation Powerhouse



Descriptive statistics

Use `.describe()` to quickly obtain count, mean, standard deviation, quartiles and more



Central tendencies

Calculate `.mean()`, `.median()` and `.mode()` to understand data centre



Dispersion metrics

Use `.std()`, `.var()`, `.min()` and `.max()` to assess data spread



Frequency analysis

Count unique values and frequencies with `.value_counts()` and `.nunique()` for categorical data

Grouping and Aggregation

Split-Apply-Combine: Core Paradigm for Grouped Data Analysis



Group data

Use `.groupby()` to segment data based on categorical variables



Apply functions

Perform aggregations (sum, mean, count) on each group



Multiple aggregations

Apply multiple operations at once using `.agg()` for detailed analysis



Pivot tables

Create cross-tabulations and summary tables using `.pivot_table()` for Excel-like functionality

Mastering Missing Data

Detect missing values

Use `.isnull()` and `.notnull()` to identify gaps in data

Smart removal

Remove incomplete records with `.dropna()` by specifying rows or columns

Strategic imputation

Use `.fillna()` to fill missing values with strategies like mean, median or custom values

Advanced interpolation

Estimate missing values using `.interpolate()` for time-series and numerical data



Data Transformation Techniques



Function application

Use `.apply()`, `.map()` and `.applymap()` to perform custom calculations and clean data



Data binning

Group continuous data into categories with `pd.cut()` or `pd.qcut()` for analysis and modeling



Vectorised operations

Apply element-wise operations efficiently using pandas' optimised computation, avoiding explicit loops



Categorical encoding

Convert categorical variables into numeric formats for machine learning and statistical tasks

Real-World Example

Customer Analytics



Data inspection

Use `.info()` to check missing values, data types and memory usage

Behavioural analysis

Analyse spending patterns with
`.groupby('gender').mean()`

Real-World Example: Customer Analytics

Customer Analytics



Geographic Insights

Count unique customer locations with `.nunique()`

Outlier detection

Identify unusual purchases using `.describe()`

Visualisation Integration

Built-in Plotting Power



Uses `.plot()` for rapid exploratory data analysis including:

- Line plots for trends
- Bar charts for comparisons
- Histograms for distributions
- Scatter plots for relationships

Integrates with Matplotlib and Seaborn for publication-quality visualisations

Key Takeaways

Comprehensive toolkit

Equip data scientists with tools for all stages of the workflow

Efficient exploratory analysis

Use summarisation functions to accelerate insights discovery

Powerful data transformation

Apply grouping, aggregation and transformation for deeper analytical insights

Powerful data transformation

Build a base for advanced analytics, machine learning and data science tasks

Hands-on: Numpy

Refer to exercise document.

Hands-on: Pandas

Refer to exercise document.

Data Visualisation with Matplotlib and Seaborn

Turning data into insights through powerful Python visualisation libraries

Data Visualisation with Matplotlib and Seaborn



Create visual stories

Transform raw numbers into visuals that drive understanding and action

Enable exploratory analysis

Use tools for pattern discovery and hypothesis generation

Importance of Data Visualisation



Clarify data

Convert complex datasets into easily interpretable visuals



Communicate insights

Present findings clearly to stakeholders for better understanding



Discover patterns

Identify trends, outliers and hidden relationships in the data



Inform decisions

Provide actionable insights to guide data-driven choices

Introduction to Matplotlib

Core 2D Plotting Library for Static, Animated and Interactive Visuals



Comprehensive control

Customise every aspect of plots precisely



Versatile output

Create static images, animations and interactive visualisations

Matplotlib

Key Functionalities



Line plots

Show trends over time and continuous relationships



Bar charts

Compare categorical data across groups



Histograms

Display data distributions and value frequencies



Scatter plots

Analyse relationships between two continuous variables

Introduction to Seaborn



Built on Matplotlib

Leverages Matplotlib while offering a simpler, user-friendly API



Beautiful defaults

Produces elegant visualisations with minimal code



Statistical focus

Is optimised for statistical plots and data analysis workflows

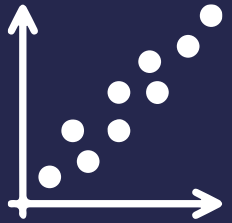


Pandas integration

Works seamlessly with DataFrames for streamlined visualisation

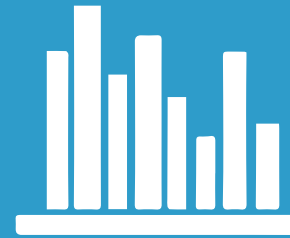
Seaborn

Key Functionalities



Distribution plots

Use histograms and KDE plots to examine data distributions and density



Categorical plots

Visualise categorical data with bar charts, box plots and violin plots



Relational plots

Explore variable relationships using scatter and line plots



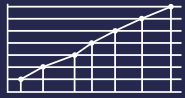
Advanced visualisations

Analyse correlations and multiple variables with heatmaps and pairplots

Matplotlib v. Seaborn

Matplotlib	Seaborn
Provides low-level, flexible control	Offers high-level, aesthetic interface
Offers detailed customisation	Focuses on statistical visualisations
Ideal for precise, custom visualisations	Ideal for quick EDA and clear insights

Combining Seaborn and Matplotlib



Start with Seaborn: Generate quick, attractive plots for initial exploration



Enhance with Matplotlib: Apply precise customisations for publication-quality visuals



Achieve synergy: Combine both libraries for a complete visualisation toolkit

Best Practices for Data Visualisation



Choose right chart type: Match visualisations to data type and analytical goals



Avoid clutter: Remove unnecessary elements and prevent misleading scales



Keep it simple: Maintain clarity and avoid complexity



Add context: Include labels, legends and titles



Use colours wisely: Enhance readability and accessibility with thoughtful color choices

Summary

Offers flexible, low-level control for custom visualisations

Provides high-level interface with statistical focus and attractive defaults

Combine both for robust data exploration and storytelling



Matplotlib

Seaborn

Complete Toolkit

Mastering Code Modularisation

Mastering Code Modularisation

“Where Can I Modularise My Work Using Functions/Libraries?”



Break large, repetitive code into smaller functions



Reuse existing code instead of rewriting it



Create libraries or modules for frequently repeated tasks



Improve readability, debugging and team collaboration

Examples of Effective Modularisation

Functions

Write a single `clean_data()` function to handle all preprocessing consistently.

Libraries

Use established libraries like `pandas` to replace manual loops with optimised, tested code.

Custom modules

Bundle reusable functions into a module such as `utils.py` and import it across projects.

Examples of Effective Modularisation

Reflection Questions



Which parts of your recent project could be modularised into functions?

What repeated tasks could be converted into a reusable library?

Q & A

Thank you